INES-RUHENGERI · FACULTY OF SCIENCES AND INFORMATION TECHNOLOGY

MyTurn

Design and Implementation of an IoT-Based Waiting Line Management and Service-Desk Status System

Your Turn, Made Simple.

TICKET PAGE | WALL DISPLAY | DESK CONSOLE | ESP32 BOARD

Leah NIYODUHORANA	25/27754
Odilla DUKUNDIMANA	25/26987
Isaac MUSHIMIYIMANA	24/25783

Internet of Things (IoT)
Lecturer: Joselyne Byukusenge
Musanze · 23 October 2026

A numbered line is still a line.

1

Waiting blind

A visitor cannot tell how long the wait is, so they crowd the service point, leave, or come back at the wrong moment.

2

One absent visitor

The desk calls a number that is not there. Without a rule that recovers, everyone behind it waits.

3

No record

A paper list cannot say how long service takes, how many visitors never answered, or who changed the desk.

> The desk needs a number, a live status, and a fair rule for the person who does not answer.

A QR code, a number, and a desk that recovers by itself.

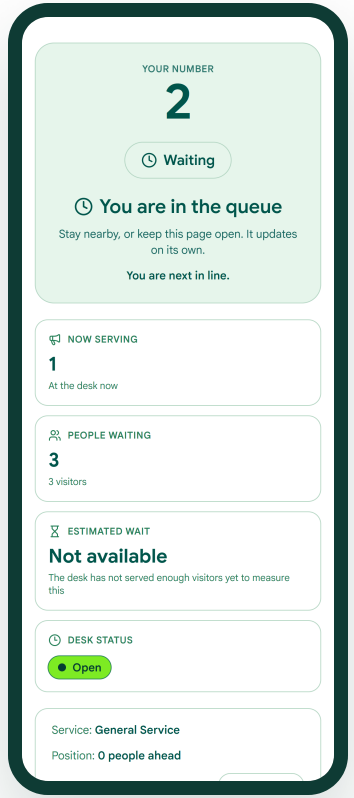
1 **Scan**
The QR code at the door opens the ticket page. No app, no account.

2 **Choose the service**
The server issues the next number of that waiting line and a private tracking link.

3 **Follow your turn**
The page shows the called number, the wait and how many are ahead.

4 **Be called**
The display and the phone update together, without a refresh.

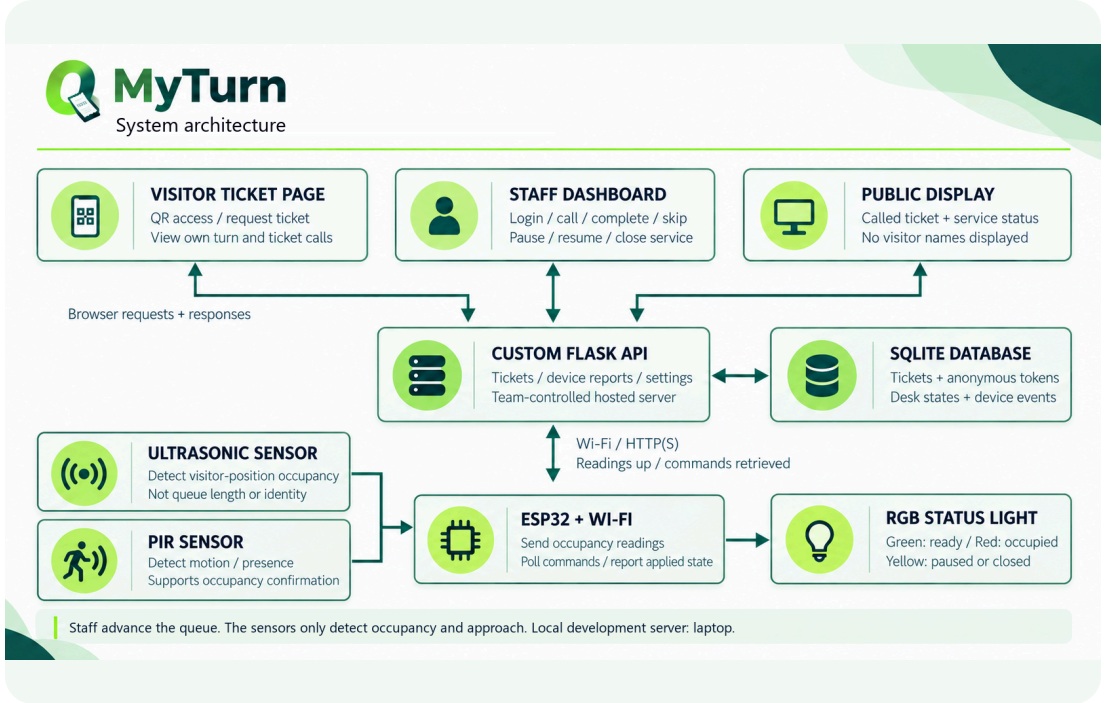
No name · no phone number · no account



The visitor page, built and running

Four parts, one waiting line.

- **The board measures**
  An ESP32 reads distance and approach, then applies the desk state to four LEDs.
- **The API decides**
  Flask owns the waiting line rules, a counter per service, the desks, the accounts and the event log.
- **SQLite remembers**
  Services, waiting lines, desks, board readings and accounts in one file, with every change written down.
- **The pages show**
  Next.js: visitor, wall display, staff console and administration.
- **One origin**
  The browser reaches the API through the same origin; the board proves itself with a device key.



ESP32 → Flask API → SQLite → Next.js

What sits at the service point.

ESP32 DevKitC	the controller
HC-SR04 ultrasonic	TRIG 5 · ECHO 21
HC-SR501 motion	GPIO 19
Four LEDs	25 · 26 · 27 · 33
Buzzer	GPIO 32

RWF 46,200 estimated prototype budget

The enclosure, the sensor mount and the power supply are part of the same budget.



Wiring at a glance · the full schematic is in the report

The sensors measure a place, not a person.

MEASURED

- ✓ Distance from the HC-SR04, turned into **occupied** or **free**.
- ✓ An approach from the PIR, held for a short window.
- ✓ The LED state the board actually applied, sent back with every report.
- ✓ A median of five distance readings, trusted only after three, so one bad echo does not change the state.
- ✓ Each reading is stored with its time, distance, occupancy, approach and applied LED state.

NEVER

- × No counting people in a line.
- × No identity, face, phone or name.
- × No camera, no microphone, no tracking.
- × No guess about who deserves priority.
- × No record of a person, a visit or a conversation.

A waiting line system does not need to know who you are. The HC-SR04 alone decides occupancy; the PIR only reports that somebody is walking up.

A bag is not a person. An object that has not moved for five minutes, with no approach seen in that window, stops blocking the call. The hold rule is not relaxed: only the call guard is.

One line per service, one call per desk.

1. **The number comes from the server**
   Each service has its own counter row, updated in the same transaction that inserts the ticket, so two requests cannot share a number.

2. **The link is opaque**
   The tracking URL carries a random token, never the ticket number or a sequential id.

3. **One call per desk**
   A desk serves one service and holds one call; two desks may work the same service at once. A per-desk unique index and a transaction enforce it.

4. **Nobody blocks the desk**
   After the hold, an unanswered number goes to the back of its own service with a fresh number.

5. **Everything is written down**
   Every change is an event with a time and an actor: a person, or the system.

A TICKET'S LIFE

waiting called → **called** completed → **completed**

called skipped → **skipped**

called cancelled → **cancelled**

called hold ends → **waiting** fresh number

recall restarts the hold; undo takes back the last decision within 10 seconds. The default hold is 60 seconds, and the next slide shows the one case where the board can shorten it.

The board can end an unanswered call early. Only with evidence.

Default hold 60 seconds



Without a fresh reading, the full hold always applies.

With evidence 25 seconds



The unused part of the hold is given back to the waiting line.

grace · service point empty · full hold · not used

✓ **Fresh board** ✓ **No occupancy** ✓ **No approach** ✓ **Past the grace** **Otherwise: full hold**

The server decides. The board only reports. An offline board, a missing reading, an occupied service point or an approach keeps the full hold, so absence of data is never treated as absence of a person.

Why a floor? A visitor who is walking in needs time to reach the desk. Ten seconds of grace, then a floor of 25, stops the rule from firing before they arrive.

A wait you can trust, or no number at all.

3

served tickets before the page offers an estimate.
Below that it says “Not available”.

MYTURN_ETA_MIN_SAMPLES, not a guess

estimate = measured average × tickets ahead

1 **Measured, not modelled**

Each sample is the time from **called_at** to **closed_at** of a ticket that was actually served.

2 **The sample count is visible**

The visitor sees the average and how many tickets it came from, so a thin average cannot look certain.

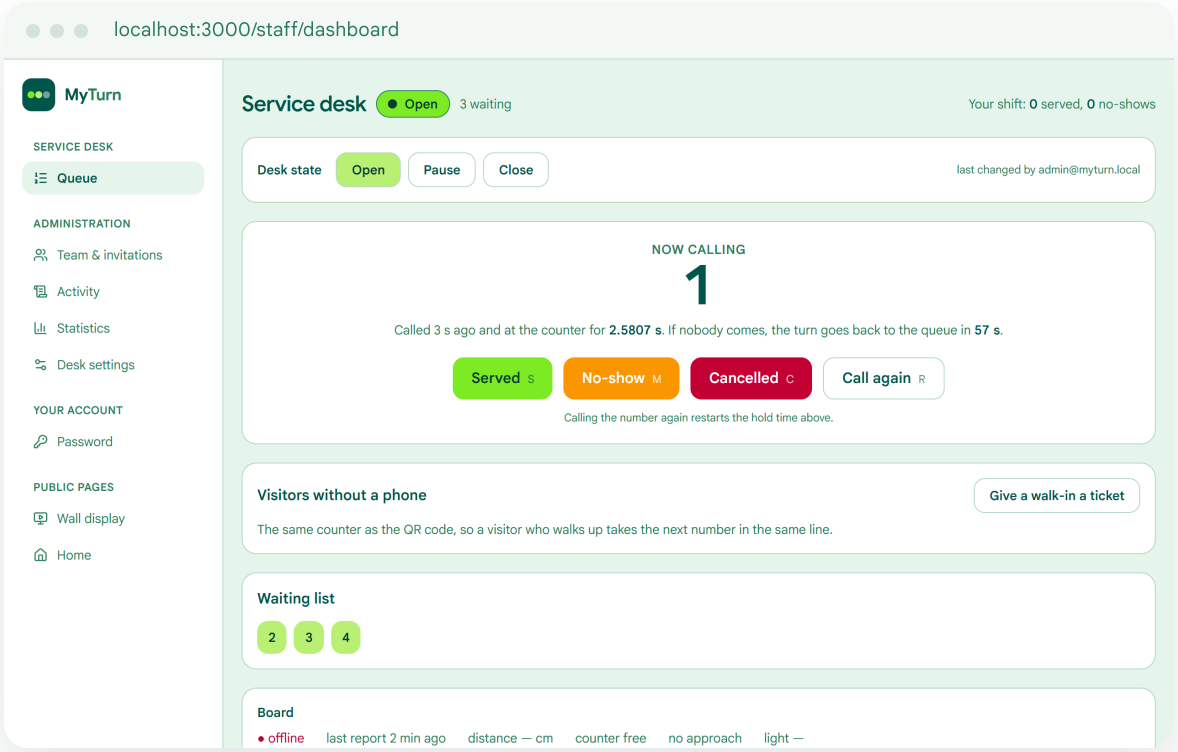
3 **Small demo times are shown**

A desk that serves in seconds reports seconds. The figure is not hidden to look better.

4 **A target is compared, not promised**

If the service sets a target, the statistics page reports how often it was actually met.

Two views of the same waiting line.



Staff: call, complete, skip, cancel, walk-in, recall and undo.

Live: hold countdown, average service time, keyboard shortcuts, per-desk analytics.

Administration

A **Team & invitations**
Invite, promote, switch off; the last administrator cannot be removed.

B **Activity**
Ticket, desk and account history in one list, with the reason for each requeue.

C **Statistics**
Counts, service-time distribution, busiest hours, target comparison and CSV.

D **Services & desks**
Add a service with its letter, edit the wording and target, and see each service point's live state.

Rights are granted, checked and recorded.

VISITOR

No account

An opaque ticket token in the link. No name, phone number or email is ever requested.

STAFF

Invitation

Single-use, 72 hours. Sign in as a person, or with the shared classroom password.

ADMINISTRATOR

Promoted

Created only by the seed script or by another administrator. Never by public registration.

DEVICE

Device key

The ESP32 sends a key with every reading. It can report and ask for a setting; it cannot change the waiting line.

Server-side roles. Every admin route is checked twice: the page, and the API.

Invitations cannot create admins. Asking for one is a 400.

No address leaks. A password reset answers the same whether or not the address exists.

CSRF on writes. A double-submit token, kept in memory, not in localStorage.

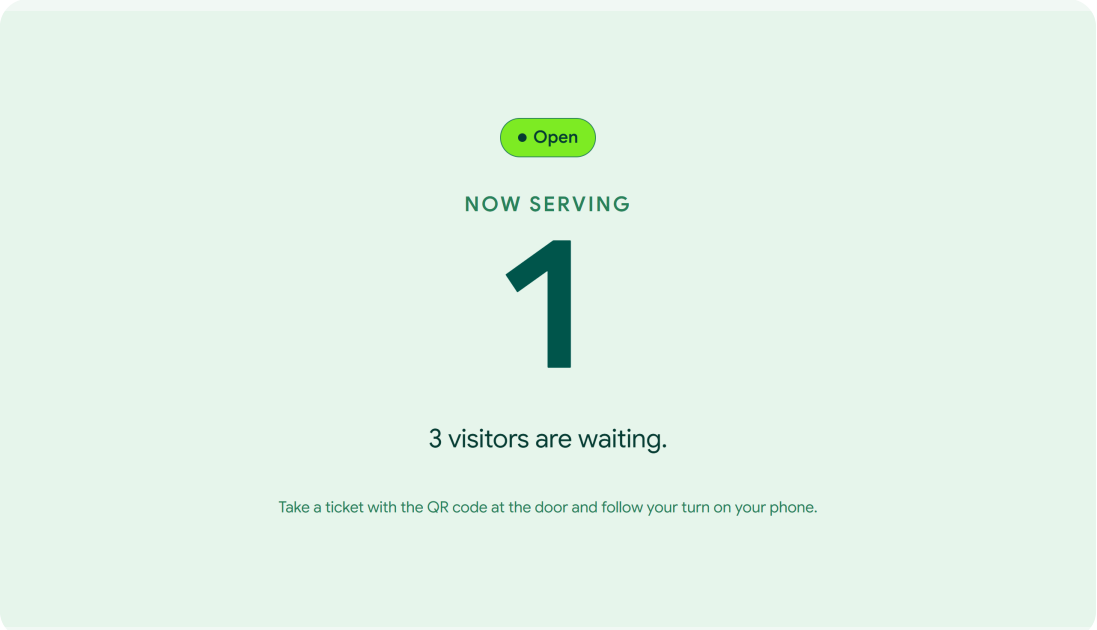
Guarded last admin. You cannot demote the final administrator or switch off your own account.

Recorded. Account changes land in the activity history with the actor and the time.

> A visitor never gets an account. A ticket asks for no name, phone number or email.

What we will show on the day.

1. **Take a ticket on a phone**
   Pick a service; the number comes from that waiting line's counter and the tracker opens.
2. **Watch the display change**
   Call next at the desk; the wall screen and the phone update together.
3. **Serve or record a no-show**
   Complete, skip or cancel. Undo is available for ten seconds.
4. **Let a call expire**
   The ticket goes to the back of its service with a fresh number, and the reason is logged.
5. **Show the board**
   Distance and approach change the LED; the dashboard shows what the board applied.
6. **Close the desk**
   The ticket page says so before a visitor joins, and the LED follows.



The waiting-area display

Tested, not assumed.

234
API checks: accounts, CSRF, hold, requeue, estimate gate, services and desks

71
browser checks on the visitor, auth and desk pages

45
console checks: activity, statistics, settings, roles

8
sidebar checks, including the shared-session rules

Every check was run against the running prototype, not read off the source.

✓ **The build is clean**
TypeScript, ESLint and the production build all pass; all sixteen pages are generated.

✓ **The rules are exercised**
Auth and CSRF, the occupancy requeue with synthetic readings, the estimate gate, undo, recall, CSV, and the board's WebSocket reporting.

! **Still measured by hand**
The physical sensor trials and the photograph of the desk rig are the remaining figures; the report leaves them blank rather than inventing them.

358 automated checks. A feature is only finished when a check proves it.

What it does not do yet.

No daily reset

The per-service counter keeps climbing across days. A midnight reset is specified but not built.

NOT BUILT

No separate SERVING state

A called ticket is treated as being served; the dwell time is the service time. This is documented, not invented.

DELIBERATE

Sensors measure only

The PIR can see a passer-by, and the ultrasonic sensor needs a clean mount. Field accuracy is still to be recorded.

TO MEASURE

No buffering on the board

An offline board keeps the full 60-second hold, by design. Readings taken while the Wi-Fi is down are discarded, not replayed.

NOT BUILT

No mail server

Invitation and reset links are shown in the app and logged in development, not emailed. A production deployment needs SMTP.

PROTOTYPE

Single process

SSE subscribers and rate-limit counters live in memory, so the API runs one worker. Scaling needs shared state.

DEPLOYMENT

Also not built: spoken announcements and SMS alerts.

Delivered, and what comes next.

DELIVERED

- ✓ A working prototype: 16 pages, 12 tables, real accounts and roles.
- ✓ The ESP32 sketch: distance, approach, four LEDs, the buzzer and the desk setting.
- ✓ Four suites, 358 automated checks, all passing.
- ✓ A proposal, a 55-page report and the design documents.

NEXT

1. Run and record the physical sensor trials; add the desk photograph.
2. Fill the measured figures into the report, without estimating them.
3. Buffer readings on the board when the Wi-Fi drops.
4. Add daily number resets for every service.
5. Speak the number aloud for visitors who cannot see the screen.

A working prototype, a tested waiting line, and a clear list of what still needs measuring.

Thank you. Questions?

Your Turn, Made Simple.